

Propósito

Provocar el escenario más adverso de búsqueda en un Árbol Binario de Búsqueda (BST) al introducir datos que ya están organizados de forma alfabética. El objetivo es mostrar cómo el BST se transforma en una lista enlazada y analizar su eficacia en comparación con un Árbol AVL mediante el conteo de pasos de búsqueda (número de comparaciones) con diferentes cantidades de datos.

1. Generación del Árbol AVL

Implementación por extensión: La clase AVL y su nodo asociado fueron desarrollados como una extensión de la clase base BST y su nodo original.

Reutilización de código: Esto facilitó la extensión de la lógica básica de búsqueda y la estructura de nodos, añadiendo solo las características propias del AVL:

Cálculo de la altura del nodo.

Determinación del factor de equilibrio (`balance_factor`).

Métodos de rotación para organizar y equilibrar automáticamente los nodos tras cada inserción

El resto de los métodos mantienen la misma funcionalidad que el BST tradicional

2. Metodología Utilizada

Ordenamiento inicial: Se tomaron los nombres de las series (SERIES) y se organizaron alfabéticamente usando `sort()`.

Carga en las estructuras: Se introdujo la misma lista organizada en ambos, el BST y el AVL.

Búsqueda y conteo: Se localizó una serie que se encontraba al final del conjunto de datos (la situación más desfavorable para el BST) y se anotó la cantidad de comparaciones/pasos hechos por cada tipo de árbol.

3. Análisis y Conclusiones

Degeneración del BST: Al recibir datos ya organizados alfabéticamente, el BST pierde su capacidad de ramificación y se transforma en una lista enlazada. Su complejidad de búsqueda se convierte en el peor caso: $O(n)$ (lineal).

Eficiencia del AVL: El árbol AVL mantiene un equilibrio estructural al realizar rotaciones

dinámicas durante el proceso de inserción. Esto permite que la complejidad de búsqueda se mantenga en $O(\log n)$ (logarítmica).

Escalabilidad: Cuando la cantidad de datos aumenta de manera exponencial, el AVL muestra una ventaja significativa. Con 1,000 elementos, el AVL completa la búsqueda en solo 12 pasos, mientras que el BST requiere 260 pasos (basados en nuestras pruebas)

Problema de Recursión en BST: Al ingresar más de 1,000 elementos de manera ordenada en un BST, se genera un RecursionError debido a la gran profundidad de las llamadas recursivas en Python.

Solución sugerida: Implementar la inserción utilizando un bucle iterativo (while loop) en lugar de recursión, o cambiar directamente al uso del árbol AVL.